DQ-Steward AI

An AI-Assisted Data Quality, Monitoring and Stewardship Automation Platform

Detailed project specification prepared for Aham Golash, targeting AI & Automation SME (Data Engineer / BI Developer) roles in MDM and DA&A.

1. Project Overview

DQ-Steward AI is an end-to-end pipeline that ingests messy master data (customers, products, suppliers), measures its quality, detects duplicates, builds golden records, and uses an LLM to help a data steward clean and standardize records. It also monitors pipeline health, raises alerts, recommends fixes for recurring incidents, and presents everything on a dashboard. The whole thing is protected by an automated test suite, including tests that check the reliability of the AI output.

Why this project? The job description asks for AI-powered MDM support, intelligent monitoring and alerting, GenAI/NLP for data quality and stewardship, automated incident resolution and recommendations, dashboards, and AI test automation. This project touches every one of those in a small, buildable form, and it builds on skills you already have (Python, PySpark/Pandas, SQL, AWS Lambda/S3, Power BI).

Honest scope note: You are not rebuilding Informatica MDM. You are building a mini version of the same ideas (profiling, match and merge, golden record, stewardship queue, data quality scorecards) so you can discuss them credibly and show working code.

2. Goals and Success Metrics

- Ingest at least 50,000 raw records from 2 to 3 sources with deliberately inconsistent formats.
- Define 15+ data quality rules across completeness, validity, consistency, uniqueness and timeliness, and publish a quality score per source and per field.
- Detect duplicates with precision above 90% on a labelled sample, and produce one golden record per entity cluster.
- Use an LLM to standardize or classify problem records, with at least 85% agreement with a hand-labelled sample of 200 records.
- Send automatic alerts when the quality score drops, a pipeline fails, or data arrives late.
- Show a before/after improvement in overall quality score (for example 62% to 94%) as the headline result.
- Reach 80%+ test coverage and run all tests automatically on every push using GitHub Actions.

3. Architecture

The platform has six layers. Data flows from top to bottom; monitoring and testing wrap around everything.

Layer	What it does	Technology
1. Ingestion	Pulls CSV files and REST API data, lands raw copies unchanged (bronze).	Python, requests, AWS S3 (or local folders), optional Lambda trigger
2. Profiling and DQ rules	Computes column statistics, runs rule checks, scores quality (silver).	Pandas / PySpark, custom rule engine or Great Expectations
3. Match and merge	Standardizes keys, finds duplicate clusters, builds golden records (gold).	RapidFuzz, recordlinkage, SQL

Layer	What it does	Technology
4. AI stewardship	LLM fixes, classifies and explains issues; low-confidence items go to a steward queue.	OpenAI or Azure OpenAI API, Pydantic validation
5. Monitoring and incidents	Tracks metrics, raises alerts, recommends resolutions from past incidents.	SQL metrics tables, Teams/Slack/email webhooks, embeddings search
6. Analytics	Dashboards for quality trends, steward workload, pipeline health.	Power BI (primary) or Streamlit

Use the medallion pattern (bronze = raw, silver = cleaned and validated, gold = golden records and metrics). It is standard in data engineering and interviewers recognise it immediately.

4. Tech Stack

Area	Choice	Notes
Language	Python 3.11	Type hints, pydantic, logging
Processing	Pandas for core; PySpark for the scale-up version	Reuse your Netflix PySpark experience
Storage	PostgreSQL (metrics, golden records) + S3 or local Parquet (bronze/silver)	Use SQLite first if setup is slow
LLM	Azure OpenAI (preferred, matches JD) or OpenAI API	Use a cheap model; cache responses to save cost
Orchestration	Simple Python scheduler or Apache Airflow / Prefect	Airflow is a bonus, not required
Dashboard	Power BI Desktop connected to PostgreSQL	Streamlit as a free fallback
Alerting	Microsoft Teams or Slack incoming webhook, SMTP email	Webhooks are free and quick
Testing	pytest, pytest-cov, Playwright for UI tests	Playwright stands in for Opkey (see Section 6.8)
DevOps	Docker, docker-compose, GitHub Actions	CI badge on your README looks great

5. Dataset

Generate your own so you control the problems. Use the **Faker** library to create about 50,000 customer records, then inject realistic defects. Optionally combine it with a public dataset such as NYC 311 requests or Kaggle customer datasets.

Defect type	How to inject it	Example
Missing values	Blank out 5-10% of email, phone, city	email = NULL
Format errors	Mix date formats, phone patterns, casing	03/14/24 vs 2024-03-14
Typos	Swap or drop characters in names and cities	Indorre, Indor
Duplicates	Create 8-12% near-duplicates with variations	Rahul Sharma vs Rahul Sharma Jr. / R. Sharma
Conflicting values	Same entity with different address across sources	Source A: Indore, Source B: Bhopal
Invalid values	Bad emails, impossible ages, wrong pincodes	abc@@gmail, age = 240
Late or missing files	Skip or delay a daily file on some days	No file on day 5

Keep a hidden *ground truth* column (true entity ID) so you can measure duplicate-detection precision and recall. This is what makes the project measurable rather than a demo.

6. Module-by-Module Design

6.1 Ingestion and Landing (Bronze)

- Write extractors for CSV drops and one REST API (you can mock it with FastAPI or use a public API).
- Land every file unchanged with metadata: source name, load timestamp, file hash, row count.
- Reject nothing at this stage; log everything. Optionally trigger processing through an S3 Event Notification and Lambda, reusing your existing serverless ETL knowledge.
- Record each run in a `pipeline_runs` table (run_id, source, status, rows_in, duration, error).

6.2 Profiling and Data Quality Rules (Silver)

Build a small rule engine where each rule is a config entry, not hard-coded logic. This shows good design and maintainability.

```
rules:
  - id: DQ001
    dimension: completeness
    field: email
    check: not_null
    severity: high
  - id: DQ007
    dimension: validity
    field: phone
    check: regex
    pattern: '^[6-9][0-9]{9}$'
    severity: medium
  - id: DQ012
    dimension: uniqueness
    field: customer_id
    check: unique
    severity: critical
```

- **Profiling:** null %, distinct count, min/max, top values, pattern frequency for every column.
- **Dimensions:** completeness, validity, consistency, uniqueness, timeliness. Score each 0-100 and compute a weighted overall score.
- **Outputs:** a `dq_results` table (rule, field, failed_count, total, run_id) and a `dq_issues` table listing each failing record with its rule ID.
- Save an HTML profiling report per run, similar to what a steward would review.

6.3 Match, Merge and Golden Record (Gold)

- **Standardize:** lowercase, trim, remove punctuation, normalise phone and address tokens before comparing.
- **Blocking:** only compare records that share a key such as the first 3 letters of the name plus the pincode. This keeps comparisons fast on 50k rows.
- **Scoring:** use RapidFuzz (Jaro-Winkler / token sort ratio) on name and address, exact match on email or phone, then combine into a weighted match score.
- **Thresholds:** above 0.90 = auto-merge; 0.70 to 0.90 = send to steward queue; below 0.70 = distinct.
- **Survivorship rules:** for each field pick the winning value using rules such as 'most recent', 'most complete', or 'trusted source wins'. Store the lineage of which source supplied each field.
- **Evaluate:** compare clusters against ground truth and report precision, recall and F1.

6.4 AI Stewardship Layer (the GenAI core)

This is the module that matters most for the job. Use the LLM only where rules are weak, and always validate its output.

- **Standardization:** convert free-text values into canonical forms, for example 'Indor', 'indore mp', 'INDORE (M.P.)' to 'Indore, Madhya Pradesh'.
- **Classification:** categorise free-text fields (for example job titles into departments, or product descriptions into categories) using a few-shot prompt.
- **Issue explanation:** generate a plain-English reason and suggested fix for each flagged record so a steward can approve quickly.
- **Structured output:** force JSON output and validate it with Pydantic. If validation fails, retry once, then route to the steward queue.
- **Confidence and guardrails:** ask the model for a confidence score; auto-apply only above a threshold (for example 0.9). Never let the LLM overwrite a value without logging the old value, new value, model, prompt version and timestamp in an `audit_log` table.
- **Cost and privacy:** deduplicate repeated values before calling the API, cache results, batch requests, and mask or avoid real PII. Use only synthetic data.

```
SYSTEM: You are a data steward assistant. Standardize the city value.
Return ONLY JSON: {"standardized": str, "state": str, "confidence": float 0-1,
                   "reason": str}
Rules: never invent a city. If unsure, set confidence below 0.5.
EXAMPLES:
  "indor mp"  -> {"standardized":"Indore","state":"Madhya Pradesh","confidence":0.97,...}
INPUT: "{raw_value}"
```

- **Stretch (NLP):** use sentence embeddings (sentence-transformers or OpenAI embeddings) to find semantically similar product names or categories that rules miss.
- **Stretch (ML):** train a small scikit-learn model (for example logistic regression or gradient boosting) on labelled pairs to predict duplicate vs not duplicate and compare it with the rule-based matcher.

6.5 Intelligent Monitoring and Alerting

- Store metrics per run in a `metrics` table: quality score, rows in/out, rule failures, latency, LLM cost, steward queue size.
- **Static alerts:** overall score below 85, a critical rule failing, a pipeline run failing, or a daily file missing by 9 AM.
- **Dynamic alerts (anomaly detection):** compare today's null % or row count to a rolling 14-day mean and flag values beyond 3 standard deviations (z-score). Optionally try Isolation Forest.
- Send alerts to Teams/Slack with a short summary, severity, affected source and a link to the dashboard page. De-duplicate alerts so the same problem does not fire repeatedly.
- Add a simple health endpoint or heartbeat table so you can show 'pipeline health' on the dashboard.

6.6 Automated Incident Resolution and Recommendation Engine

Every alert becomes an incident record. The system then tries to resolve or advise.

- **Runbook automation (rule-based):** map known failure patterns to actions, for example 'missing file' triggers a retry of the extractor; 'schema drift' triggers a schema-compare report; 'null spike in one field' triggers re-run of the standardization step.
- **Recommendation engine (AI-based):** embed past incident descriptions and their resolutions. For a new incident, retrieve the 3 most similar past incidents (cosine similarity) and have the LLM draft a recommended fix and steps, citing the past incident IDs. This is a small retrieval-augmented generation (RAG) pattern.
- Track `time_to_detect` and `time_to_resolve`, and show how much auto-resolution reduced manual work.
- Keep a human in the loop: auto-actions only for low-risk fixes; everything else is a recommendation.

6.7 Dashboards and Operational Analytics

Page	Visuals	Audience
Data Quality Scorecard	Overall score gauge, score by dimension, trend over time, top 10 failing rules, before vs after cleansing	MDM / DA&A stakeholders
Steward Workbench	Queue size, items by priority, approval rate of AI suggestions, average handling time, duplicates awaiting review	Data stewards
Pipeline Health	Run status, latency, rows processed, SLA breaches, open alerts and incidents	Data engineers
AI Performance	AI acceptance rate, confidence distribution, cost per 1,000 records, error/validation-failure rate	Product owner

Build in Power BI to match your skills and the BI part of the role, and take screenshots for the README. If Power BI is not possible on your machine, build the same pages in Streamlit.

6.8 Test Automation (AI test automation, Opkey-style)

Opkey is a commercial no-code test automation platform, so you probably cannot use it for free. The honest approach is to build the same discipline with open tools and then **learn how Opkey works** (watch its public demos and docs) so you can speak about the difference. Say this in interviews: it shows maturity.

- **Unit tests (pytest):** every DQ rule, standardizer and survivorship function, with edge cases.
- **Data tests:** row-count reconciliation between bronze, silver and gold; no duplicate primary keys in gold; schema checks.
- **AI output tests:** a fixed golden set of 200 labelled inputs; assert accuracy stays above 85% for the current prompt and model. Run it whenever a prompt changes, like a regression suite for the LLM. Also test that invalid JSON and low-confidence outputs are handled correctly (use mocked responses).
- **UI tests (Playwright):** automate the Streamlit/steward interface: load page, approve a suggestion, verify the record updates.
- **CI/CD:** a GitHub Actions workflow runs lint, tests and coverage on every push and blocks the merge if tests fail.
- **Optional:** use the LLM to generate additional test cases from the rule definitions, then review them manually. That is a real 'AI for test automation' use case.

7. Database Design

Table	Key columns
pipeline_runs	run_id, source, started_at, ended_at, status, rows_in, rows_out, error_msg
dq_rules	rule_id, dimension, field, check_type, params, severity, active
dq_results	run_id, rule_id, failed_count, total_count, score
dq_issues	issue_id, run_id, record_id, rule_id, status, assigned_to
match_pairs	pair_id, record_a, record_b, match_score, decision (auto / review / distinct)
golden_records	golden_id, customer fields, source_lineage (JSON), created_at, updated_at
steward_queue	task_id, type, payload, ai_suggestion, ai_confidence, status, resolved_by, resolved_at
audit_log	id, record_id, field, old_value, new_value, changed_by (rule/AI/human), model, prompt_version, ts
metrics	run_id, metric_name, metric_value, ts
alerts / incidents	incident_id, type, severity, detected_at, resolved_at, resolution, similar_incident_ids

8. Suggested Repository Structure

```
dq-steward-ai/
  README.md              (architecture diagram, results, screenshots)
  docker-compose.yml
  config/
    rules.yaml           (DQ rules)  |  thresholds.yaml  |  prompts/
  src/
    ingestion/           extractors, S3/Lambda handlers
    profiling/           profiler, rule_engine
    matching/            standardize, blocking, scoring, survivorship
    ai/                  llm_client, standardizer, classifier, explainer, rag
    monitoring/          metrics, anomaly, alerts
    incidents/           runbooks, recommender
    db/                  models, migrations
  dashboards/            powerbi/ (.pbix + screenshots), streamlit_app.py
  tests/
    unit/  data/  ai_eval/  ui/
  data/
    generate_synthetic.py
  .github/workflows/ci.yml
```

9. Four-Week Build Plan

Week	Focus	Deliverables
1	Foundation: data generation, ingestion, profiling, DQ rule engine, PostgreSQL schema	Synthetic dataset with injected defects; bronze and silver layers; first quality scorecard; unit tests for rules
2	Match and merge, golden records, AI stewardship (standardize, classify, explain), audit log	Duplicate detection with precision/recall; LLM module with JSON validation and confidence routing; steward queue table
3	Monitoring, anomaly alerts, incident table, recommendation engine, Power BI dashboards	Teams/Slack alerts working; RAG-style recommender; 4 dashboard pages with screenshots
4	Test automation, CI/CD, Docker, documentation, polish	pytest + Playwright + AI evaluation suite; GitHub Actions badge; README with architecture diagram, results and a 2-3 minute demo video

If time is short, protect the core: **DQ rules, duplicate detection, LLM standardization with validation, alerts, one dashboard, tests**. Everything else is a stretch goal.

10. How This Maps to the Job Description

JD requirement	Where it appears in the project
AI-powered solutions for MDM operations	Match/merge, golden record, steward queue (6.3, 6.4)
Identify automation opportunities	Rule-driven pipeline, runbook automation, reduction in manual steward effort (6.6)
Intelligent monitoring and alerting	Static and anomaly-based alerts, health metrics (6.5)
GenAI / ML / NLP for data quality and stewardship	LLM standardization and classification, embeddings, optional ML matcher (6.4)
Automated incident resolution and recommendations	Runbooks and RAG recommender (6.6)
Dashboards and analytics	Four-page Power BI dashboard (6.7)
AI test automation (Opkey or similar)	pytest, Playwright, LLM regression suite, CI (6.8)
Python, APIs, workflow automation, cloud	Python modules, REST ingestion, S3/Lambda or Azure equivalent, Docker
Documentation and best practices	README, design notes, runbooks, audit trail, config-driven rules

11. Resume Bullets (use only after you have built it)

- Built an AI-assisted data quality and stewardship platform in Python that profiles 50K+ master-data records, applies 15+ configurable DQ rules and raises the overall quality score from X% to Y%.
- Implemented fuzzy duplicate detection and golden-record survivorship logic, achieving X% precision and Y% recall against labelled ground truth.
- Integrated Azure OpenAI/OpenAI for record standardization and classification with Pydantic validation, confidence-based routing and a full audit log, reaching X% agreement with human-labelled data.
- Designed anomaly-based monitoring and Teams/Slack alerting plus an embedding-based incident recommendation engine, cutting simulated time-to-resolve by X%.
- Automated testing with pytest, Playwright and an LLM regression suite in GitHub Actions CI, with 80%+ coverage.

Replace X and Y with real numbers from your own runs. Never claim a figure you did not measure.

12. Interview Talking Points

- Why you used rules first and the LLM only for ambiguous cases (cost, speed, determinism).
- How you prevent hallucinations: structured output, validation, confidence thresholds, human approval, audit log.
- How match and merge works: blocking, similarity scoring, thresholds, survivorship rules, precision vs recall trade-off.
- The five data quality dimensions and how you scored them.
- How you tested non-deterministic AI output with a fixed labelled set and thresholds.
- What you would change for production: Azure services (Data Factory, Azure OpenAI, Key Vault, Azure Monitor), real MDM tools such as Informatica CLAIRE, CDI and CDQ, access controls and PII handling.

13. Common Pitfalls to Avoid

- Do not use real personal data; keep everything synthetic and keep API keys in environment variables, never in GitHub.
- Do not send all 50,000 rows to the LLM. Deduplicate values, cache, and batch.
- Do not skip measurement. A project without numbers (precision, recall, accuracy, score improvement) is far less convincing.
- Do not over-build the UI. Backend quality, tests and documentation matter more for a data engineering role.
- Do not claim Opkey or Informatica experience. Say you built equivalent capabilities and have studied those tools.

14. Learning Resources (search these terms)

- MDM basics: golden record, match and merge, survivorship, data stewardship, data quality dimensions.
- Libraries: RapidFuzz, recordlinkage, Great Expectations, Pydantic, Faker, Playwright, pytest-cov.
- Informatica: read the public overview pages for CLAIRE AI, Cloud Data Quality (CDQ) and Cloud Data Integration (CDI).
- Azure: Azure OpenAI quickstart and Azure Data Factory fundamentals (free Microsoft Learn modules).
- Opkey: public product pages, demo videos and documentation on AI-driven test automation.